STM32基础 - SPI通信

课程目标

1. 熟悉SPI通信协议。
2. 熟悉W25Q64，理解其内部工作原理。
3. 熟悉STM32的SPI外设工作过程，能够利用I2C协议控制W25Q64。

课程试验

1. 利用STM32完成软件SPI读写W25Q64试验
2. 利用STM32完成硬件SPI读写W25Q64试验

课堂引入

SPI通信和刚刚学过的I2C通信差不多，两个协议的设计目的都一样的。都是实现主控芯片和各种外挂芯片之间的数据交流，有了数据交流的能力。主控芯片就可以挂载并操纵各式各样的外部芯片，来实现一个功能更加强大的控制系统。相对来说I2C在硬件资源配置上更节约一些，这样也就会牺牲一定的性能。而SPI在硬件配置上留有冗余，性能上比I2C要高不少。相对来说I2C是半双工通信，而SPI属于全双工通信，因此在高速性能上I2C比不上SPI，所以SPI更适合于应用在EEPROM、FLASH、实时时钟、AD转换器、数字信号处理器和数字信号解码器等的通信。

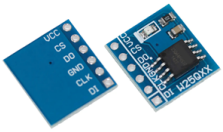
授课进程

一、SPI通信

1、基本介绍

SPI（Serial Peripheral Interface），也叫串行外设接口，注意和前面讲过的USART串口不是一回事。SPI是由Motorola公司开发的一种通用数据总线，和I2C一样都是通用的数据通信总线。同时它们也都是用于主控和外挂芯片之间的通信，应用领域也非常相似。当然SPI和I2C也是各有优势的。通过前面的I2C的学习可以发现无论是硬件电路还是软件时序设计，I2C都是相对比较复杂的。硬件上需要配置为开漏外加上拉的模式，一根通信线兼具数据收发，应答位的收发，寻址机制的设计等等。通过这么多的设计使得I2C通信的性价比非常高。I2C可以在消耗最低硬件资源的情况下，实现最多的功能。在硬件上无论挂载多少个设备，都只需要两根通信线。在软件上数据双向通信，应答位都可以实现。但是I2C由于开漏外加上拉电阻的电路结构，使得通信线高电平的驱动能力比较弱，这就会导致通信线由低电平变到高电平的时候，上升沿耗时比较长，这会限制I2C的最大通信速度。所以I2C的标准模式只有100KHz的时钟频率，I2C的快速模式也只有400KHz。即使现在还有通过改进电路的方向，设计出来高速模式可以达到3.4MHz。但是高速模式目前普及程度并不是很高。所以一般情况下，大多认为I2C的时钟速度最多就是400KHz这个速度。相比较SPI而言，还是慢了很多的。SPI传输更快，SPI协议并没有严格规定最大传输速度。这个最大传输速度取决于芯片厂商的设计需求，比如后面要学习的W25Q64存储器芯片，其**SPI时钟频率最大可达80MHz，比STM32F1的主频还要高。**

SPI的设计比较简单粗暴，实现的功能没有I2C那么多。SPI的硬件开销也比较大，它有四根通信线：**SCK**（Serial Clock）串行时钟线、**MOSI**（Master Output Slave Input）、**MISO**（Master Input Slave Output）、**SS**（Slave Select）从机选择线。SPI的基本特征是同步全双工，SCK就是用来做时钟同步的，而全双工指的就是MOSI、MISO一路可以用来发送数据，而另一路就可以用来接收数据，这个发送和接收的动作由于硬件有两根线，完全可以同时进行，两路互不影响。SPI支持总线挂载多设备，但是SPI只支持一主多从，不支持多主机。这也可以看出SPI的功能没有I2C丰富。



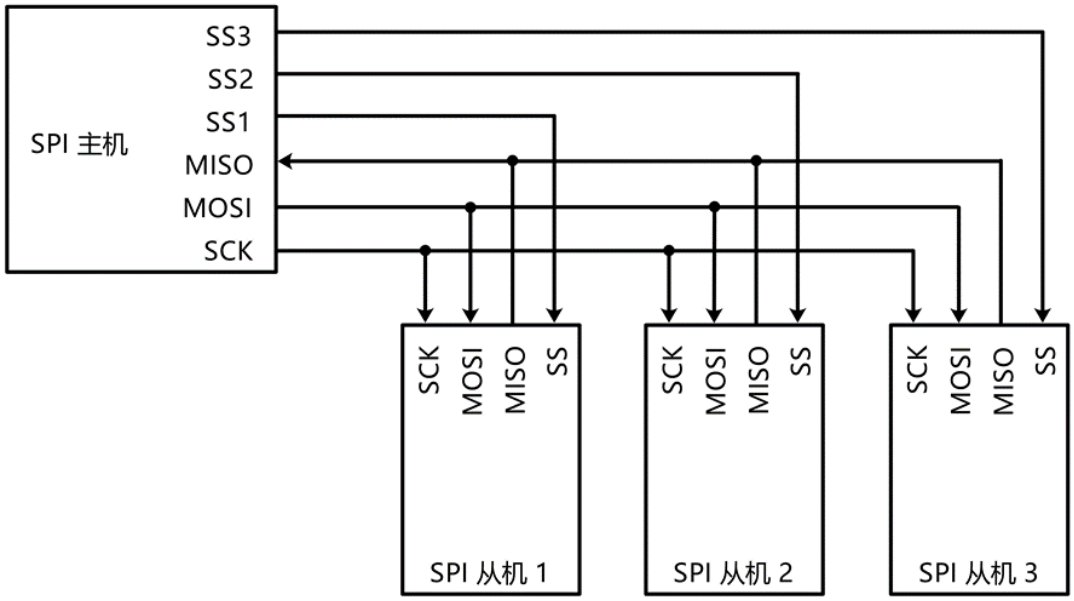




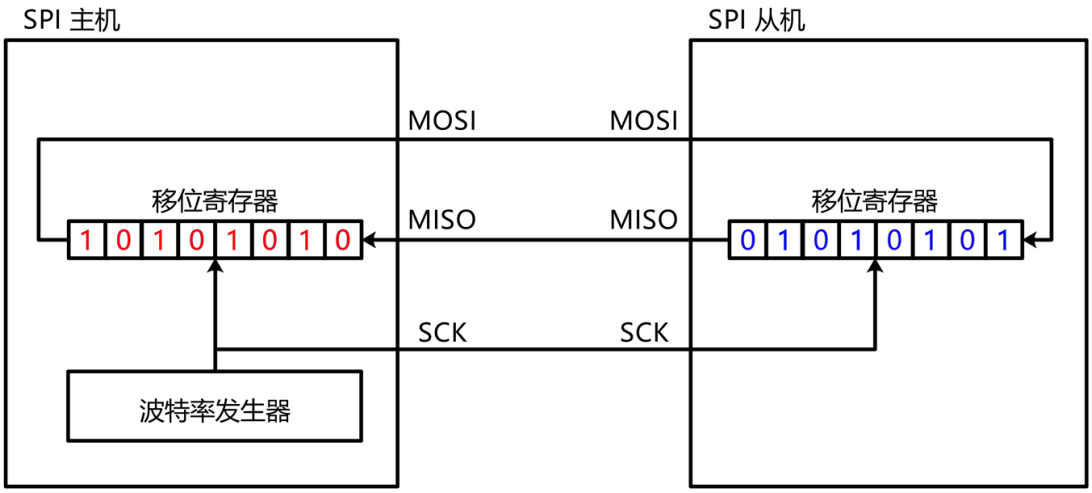
上图都是能够支持SPI的模块或设备。左边第一个是W25Q64模块，它是一个Flash存储器。这个模块的引脚和SPI定义的名字略有区别，其中CLK就是SCK，DI和DO就是MOSI和MISO。这个芯片只能作为从机，因此其DI就是从机的数据输入SI，对应就需要接在主机的MOSI引脚上，而另一个DO就是MISO了。一般在这种始终作为从机的设备上，可能会用DI和DO的简写，像STM32这种可以进行身份转换的设备，一般都会把MOSI和MISO全称写完整。还有一个CS片选，其实就是SS从机选择。左边第二个是利用SPI通信的OLED屏幕，上面的引脚也不是标准的名称。第三个是一个2.4G无线通信模块，芯片型号是NRF24L01，这个芯片使用的就是SPI通信协议，要想使用这个芯片来进行无线通信，那就需要利用SPI来读写这个芯片。最后一个就是常见的Micro SD卡，这个SD卡官方的通信协议是SDIO，但是它也是支持SPI协议的，我们可以利用SPI对这个SD卡进行读写操作。

2、硬件电路

所有SPI设备的SCK、MOSI、MISO分别连在一起。主机另外引出多条SS控制线，分别接到各从机的SS引脚，输出引脚配置为推挽输出，输入引脚配置为浮空或上拉输入。



3、移位原理图



把这个移位示意图搞懂了，那无论是前面的硬件电路，还是等会要学习的软件时序，理解起来都会更加轻松。在SPI的基本收发电路，就使用了这样一个移位的模型。左边是SPI主机，里面有一个8位的移位寄存器，右边是SPI从机，里面也有一个8位的移位寄存器。这里移位寄存器有一个时钟输入端，因为SPI一般都是高位先行的，所以每来一个时钟，移位寄存器就会向左进行移位，从机中的移位寄存器也是同理。然后移位寄存器的时钟源是由主机提供的，这里叫做波特率发生器。它产生的时钟驱动主机的移位寄存器进移位，同时这个时钟也通过SCK引脚进行输出，接到从机的移位寄存器里，之后上面移位寄存器的接法是主机移位寄存器左边移出去的数据，通过MOSI引脚输入到从机移位寄存器的右边，从机移位寄存器左边移出去的数据，通过MISO引脚输入到主机移位寄存器的右边，这样组成一个圈。

接下来一起来看一下这个电路是如何工作的？首先规定波特率发生器时钟的上升沿，所有移位寄存器向左移动，移位移出去的位放到引脚上，波特率发生器时钟的下降沿，引脚上的位，采样输入到移位寄存器的最低位。接下来，假设主机有个数据10101010要发生送到从机，同时，从机有个数据01010101要发送到主机。此时可以驱动时钟先产生一个上升沿，这时所有的位就会往左移动一次，从最高位移出去的数据就会放到通信线上，数据放到通信线上，实际上是放到了输出数据寄存器。可以看到此时MOSI数据是1，所以MOSI的电平就是高电平，MISO的数据是0，所以MISO的电平就是低电平。这是第一个时钟上升沿执行的结果，就是把主机和从机中移位寄存器的最高位分别放到MOSI和MISO的通信线上，这就是数据的输出。接着时钟继续运行上升沿，之后下一个边沿就是下降沿，在下降沿时，主机和从机内都会进行数据采样输入，也就是MOSI的1会采样输入到从机的最低位。MISO的0会采样输入到主机的最低位，这是第一个时钟结束后的现象。时钟继续运行下一个上升沿，同样的操作，移位输出，主机现在的最高位也就是原始数据的次高位，输出到MOSI，从机现在的最高位，输出到MISO，随后下降沿，数据采样输入，MOSI数据到从机移位寄存器的最低位，MISO数据到主机移位寄存器的最低位。之后时钟继续运行，第三个时钟开始，上升沿，移位，主机输出，从机输出，下降沿采样，主机输入，从机输入，之后第四个时钟，第五个时钟……一直到第八个时钟都是同样的过程，最终8个时钟之后，会看到原来主机的10101010跑到从机里了，而原来从机里的1010101跑到主机里了，这样就实现了主机和从机一个字节的数据交换。

SPI的数据收发都是基于字节交换这个基本单元来进行的。当主机需要发送一个字节，并且同时需要接收一个字节时，就可以执行一下字节交换的时序，这样主机要发送的数据跑到从机，主机要从从机接收的数据跑到主机，这就完成了发送同时接收的目的。如果只想发送不想接收的话，也很简单。仍然调用交换字节的时序，发送同时接收，只是这个接收到的数据，不看它就行了。如果只想接收不想发送，同理还是调用交换字节的时序，发送同时接收，只是会随便发送一个数据，只要能把从机的数据置换过来就行了，最后读取置换过来的数据就完成接收了，至于随便发过去的数据，从机也不会去看它。当然，这个随便数据并不会真的随便发。一般在接收的时候，会统一发送0x00或0xFF，去跟从机换数据。

以上就是SPI的基本原理。总结一下，就是SPI通信的基础是交换一个字节。有了交换一个字节，就可以实现发送一个字节，接收一个字节，和发送同时接收一个字节这三种功能。从这些可以看出，SPI在只执行发送或只执行接收的时候，会存在一些资源浪费的现象。但是全双控的通信本来就会有浪费的情况发生，对于SPI来说这个浪费无所谓。

4、SPI时序基本单元

起始条件：SS从高电平切换到低电平。

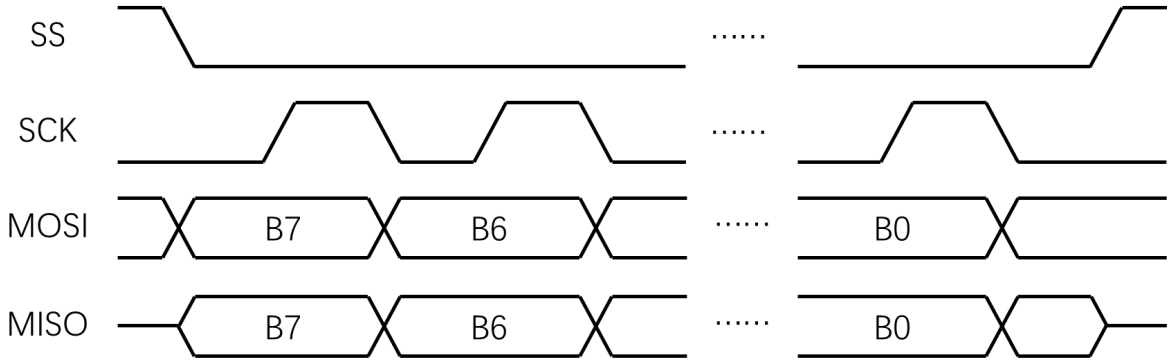
终止条件：SS从低电平切换到高电平。



（1）交换一个字节（模式0）

CPOL=0 （ 时钟极性 Clock polarity)：空闲状态时，SCK为低电平。

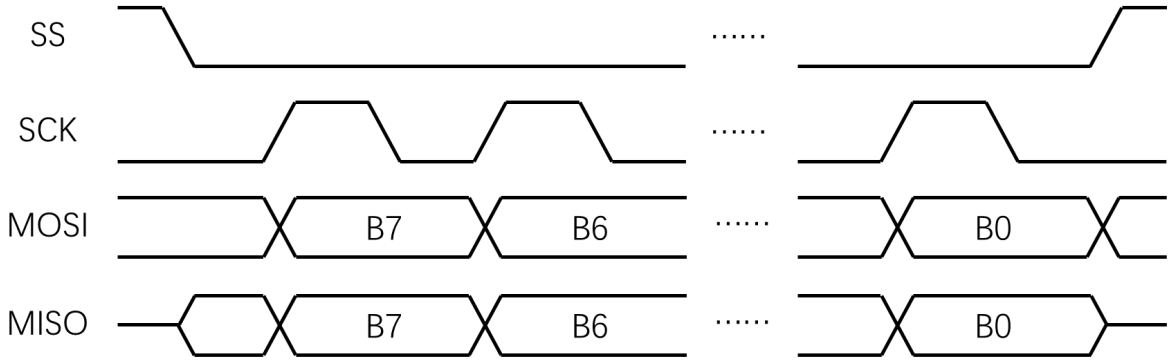
CPHA=0（时钟相位Clock phase）：SCK第一个边沿移入数据，第二个边沿移出数据。



（2）交换一个字节（模式1）

CPOL=0：空闲状态时，SCK为低电平。

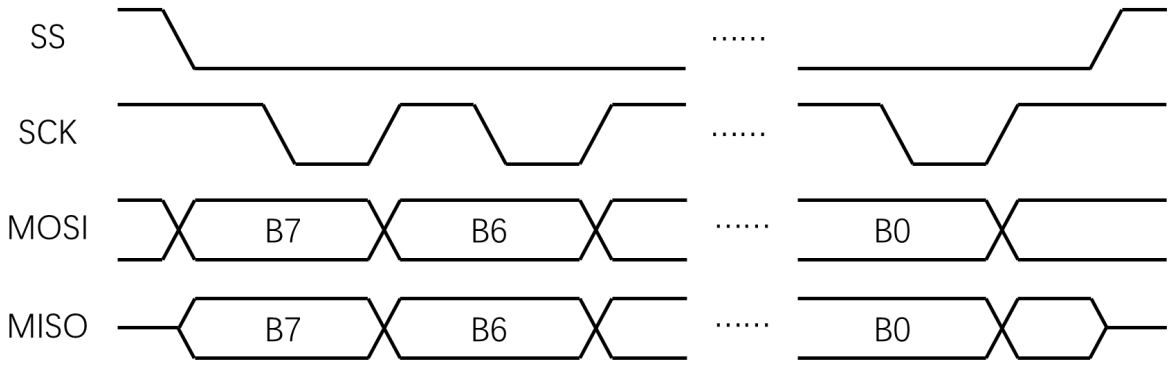
CPHA=1：SCK第一个边沿移出数据，第二个边沿移入数据。



（3）交换一个字节（模式2）

CPOL=1：空闲状态时，SCK为高电平。

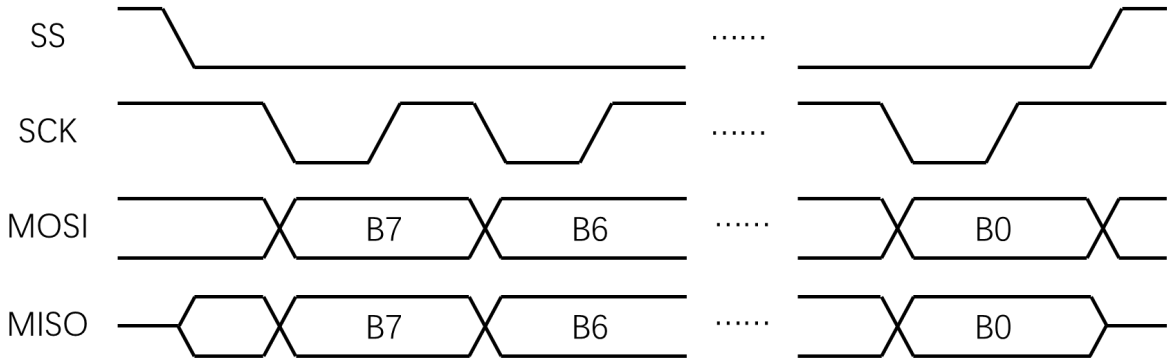
CPHA=0：SCK第一个边沿移入数据，第二个边沿移出数据。



（4）交换一个字节（模式3）

CPOL=1：空闲状态时，SCK为高电平。

CPHA=1：SCK第一个边沿移出数据，第二个边沿移入数据。

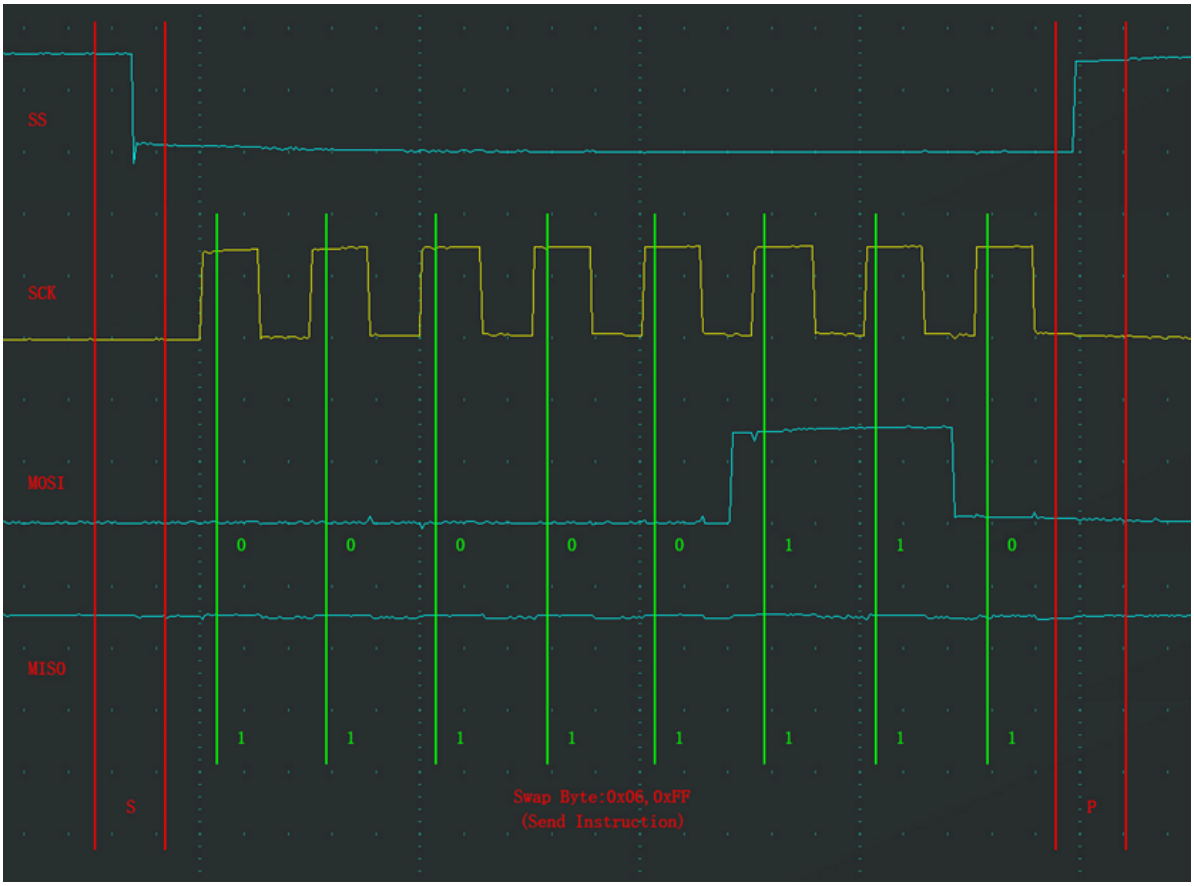


5、SPI时序

每个芯片对SPI时序字节流功能的定义不一样，下面以W25Q64芯片的时序为例看一下。SPI对字节流功能的规定不像I2C那样，I2C的规定一般是有效数据流第一个字节是寄存器地址，之后依次是读写的数据，使用的是读写寄存器的模型。而在SPI中，通常采用的是指令码加读写数据的模型。这个过程就是SPI起始后，第一个交换发送给从机的数据，一般叫做指令码。在从机中对应的会定义一个指令集，当我们需要发送什么指令时，就可以在起始后第一个字节发送指令集里面的数据，这样就能指导从机完成相应的功能。不同的指令可以有不同的数据个数。有的指令只需要一个字节的指令码就可以完成，比如，W25Q64的写使能，写失能等指令。而有的指令，后面就需要再跟要读写的数据，比如，W25Q64的写数据，读数据等。写数据，指令后面就得跟上写的位置和写什么数据。读数据，指令后面就得跟读的位置和读什么数据。这就是指令码加读写数据的模型。在SPI从机的芯片手册里，都会定义好指令集。什么指令对应什么功能，什么指令后面得跟上什么数据这些内容。

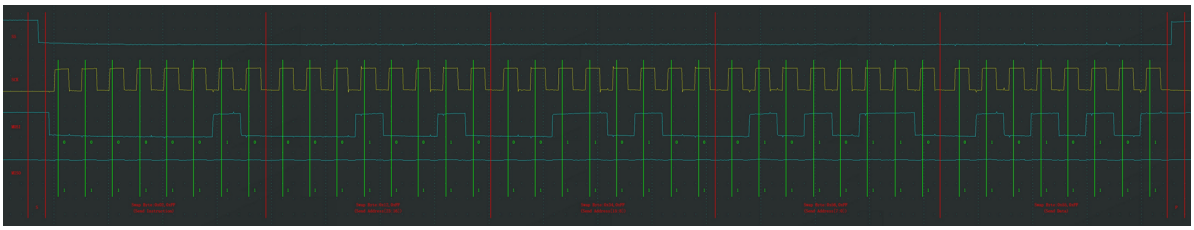
（1）发送指令

向SS指定的设备，发送指令（0x06）。



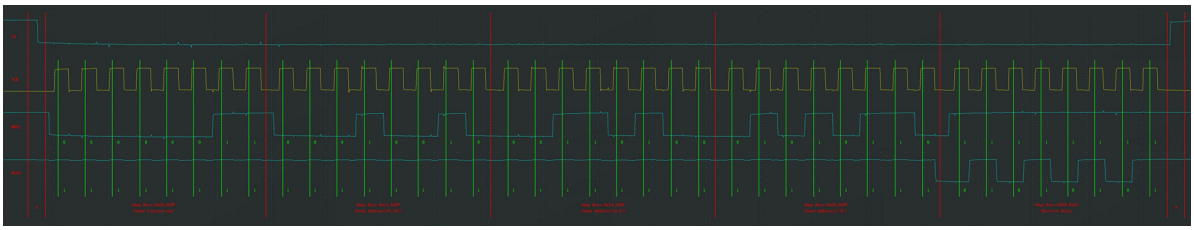
（2）指定地址写

向SS指定的设备，发送写指令（0x02），随后在指定地址（Address[23:0]）下，写入指定数据（Data）。



（3）指定地址读

向SS指定的设备，发送读指令（0x03），随后在指定地址（Address[23:0]）下，读取从机数据（Data）。



二、W25Q64

1、简介

W25Qxx系列是一种低成本、小型化、使用简单的非易失性存储器，常应用于数据存储、字库存储、固件程序存储等场景。这里xx是一个数字，不同的数字表示了这个芯片的不同存储容量。同一系列的不同型号的芯片它们的操作方法都是一样的。这个芯片是一种非易失性存储器。存储器分为易失性存储器和非易失性存储器。易失性存储器一般就是SRAM、DRAM等，非易失性存储器一般就是E2PROM、Flash等。他们最主要的区别简而言之就是存储的数据是否是掉电不丢失。非易失性存储器就是数据不容易失去的存储器。由于数据掉电不丢失，所以存储在这个芯片里的数据在断电重启后数据仍然能够保持原样。这个芯片的存储介质是Nor Flash，Flash又叫闪存存储器。闪存一般分为Nor Flash和Nand Flash，两者各有优势和劣势。这个芯片的时钟频率：80MHz / 160MHz (Dual SPI) / 320MHz (Quad SPI)。其存储容量（24位地址）包含下面这些：

W25Q40： 4Mbit / 512KByte

W25Q80： 8Mbit / 1MByte

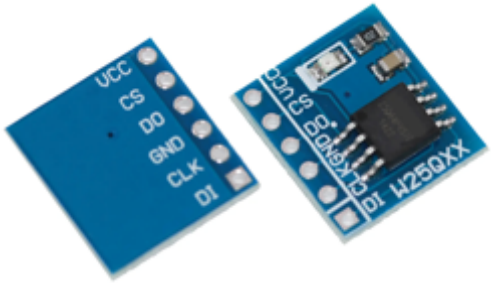
W25Q16： 16Mbit / 2MByte

W25Q32： 32Mbit / 4MByte

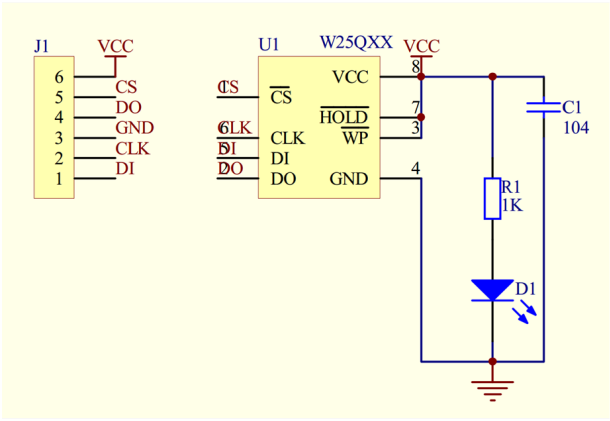
W25Q64： 64Mbit / 8MByte

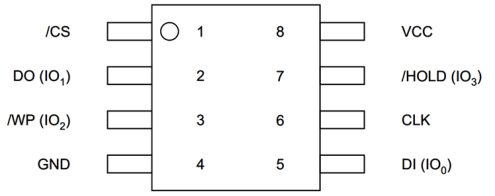
W25Q128： 128Mbit / 16MByte

W25Q256： 256Mbit / 32MByte



2、硬件电路





引脚	功能
VCC、GND	电源（2.7~3.6V）
CS（SS）	SPI片选
CLK（SCK）	SPI时钟
DI（MOSI）	SPI主机输出从机输入
DO（MISO）	SPI主机输入从机输出
WP	写保护
HOLD	数据保持

左边的图片是模块的原理图，右上角这个图是芯片的引脚定义，右下角的表就是每个引脚定义的功能。首先看一下引脚定义，8脚VCC脚，4号GND脚是电源供电。引脚供电电压是2.7-3.6伏，是一个典型的3.3伏供电设备，不能直接接入5伏电压。然后1号脚/CS，这里/代表是低电平有效，如果这里CS上面画了个横线，也是表示低电平有效。这里CS对应之前讲SPI的名称就是SS，意思是SPI的片选引脚。然后6号脚CLK对应就是SCK，是SPI的时钟线。然后5号脚DI对应MOSI，是SPI主机输出从机输入引脚。2号脚DO对应MISO，是SPI主机输入从机输出。这四个引脚就是SPI通信的四个引脚。然后这个芯片还有两个引脚，3号WP它的意思是写保护，配合内部的寄存器配置，可以实现硬件的写保护。写保护低电平有效，WP接低电平，保护住不让写；WP接高电平，不保护可以写。最后7号HOLD引脚，意思就是数据保持，低电平有效。如果在进行正常读写时，突然产生中断，然后想用SPI通信线去操控其他器件。这时如果把CS置回高电平，那时序就终止了。但如果既不想终止总线，又想操作其他器件。这可以给Hold引脚置低电平，这样芯片就Hold住了，芯片释放总线，但是芯片持续也不会终止，它会记住当前的状态。当操作完其他器件时，可以回过来将Hold置回高电平，然后继续Hold之前的时序，相当于SPI总线进来一次中断，并且在中断里还可以用SPI干别的事情，这就是Hold的功能。注意在DI、DO、WP和Hold旁边都有括号，写了IO0、IO1、IO2、IO3，这个主要对应双重SPI和四重SPI。如果是普通的SPI模式，括号里信息的都不用管；如果是双重SPI，那DI和DO就变成IO0和IO1，也就是数据同时收和同时发的两个数据位；如果是四重SPI，那就再加上WP当作IO2，Hold当作IO3，这四个引脚都作为数据收发引脚，一个时钟4个数据位。这些就是这个芯片的所有引脚功能定义了。

再看一下模块原理图。图中U1就是W25Qxx的芯片，J1是一个6脚的排针。芯片的VCC电源正极，通过VCC引脚编号接到排针6号脚，芯片GND电源负极，通过GND编号接到排针3号角，然后芯片SPI通信的四个脚，就直接通过排针引出来就行了。之后Hold和WP这两个都是直接接到了VCC，低电平有效，都接到VCC
表示这两功能都不用。然后C1电容直接接到VCC和GND，显然是一个电源滤波电容。R1和D1也是直接接到VCC和GND，在这里是一个电源指示灯，通电就亮。这些就是这个芯片的硬件电路了，接线整体上也是非常的简洁。VCC、GND接3.3伏电源，SPI通信线直接接到STM32的SPI通信引脚，Hold和WP需要用的话就接到STM32的GPIO，不需要用就直接接VCC，然后电源指示灯和电源滤波这些是都看需求加就行。

3、W25Q64框图

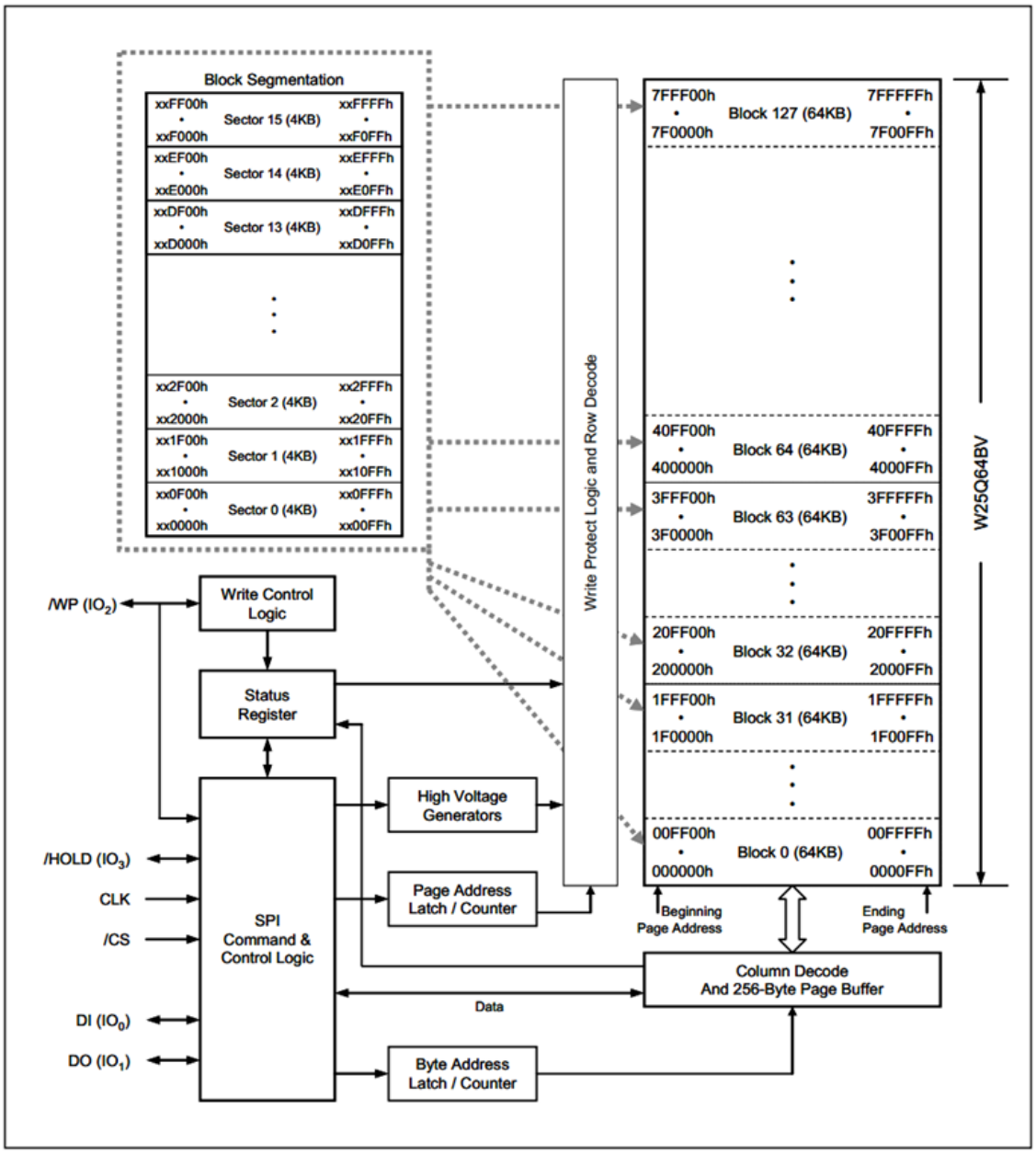


Figure 2. W25Q64BV Serial Flash Memory Block Diagram

4、Flash操作注意事项

写入操作时:

> 写入操作前，必须先进行写使能。
>
> 每个数据位只能由1改写为0，不能由0改写为1。
>
> 写入数据前必须先擦除，擦除后，所有数据位变为1。
>
> 擦除必须按最小擦除单元进行。
>
> 连续写入多字节时，最多写入一页的数据，超过页尾位置的数据，会回到页首覆盖写入。
>
> 写入操作结束后，芯片进入忙状态，不响应新的读写操作。

读取操作时:

> 直接调用读取时序，无需使能，无需额外操作，没有页的限制
>
> 读取操作结束后不会进入忙状态，但不能在忙状态时读取。

5、操作流程

读：

1. 判断芯片是否处于忙状态
2. 发送0x05,然后再发0x01读状态寄存器1的busy位，就是状态寄存器的最低位
3. 如果不忙就可以读数据
    1. 发0x03读数据
    2. 发三字节地址
    3. 接收数据
    4. 拉高cs
4. 如果忙，就等着

写：

1. 判断芯片是否处于忙状态
2. 发送0x05读状态寄存器，同时判断BUSY位和WEL位
3. 如果不忙且可以写
    1. 发0x06写使能
    2. 发0x02表示要写数据
    3. 发三字节地址
    4. 发n个数据出去
    5. 结束拉高cs
4. 如果忙，就等着

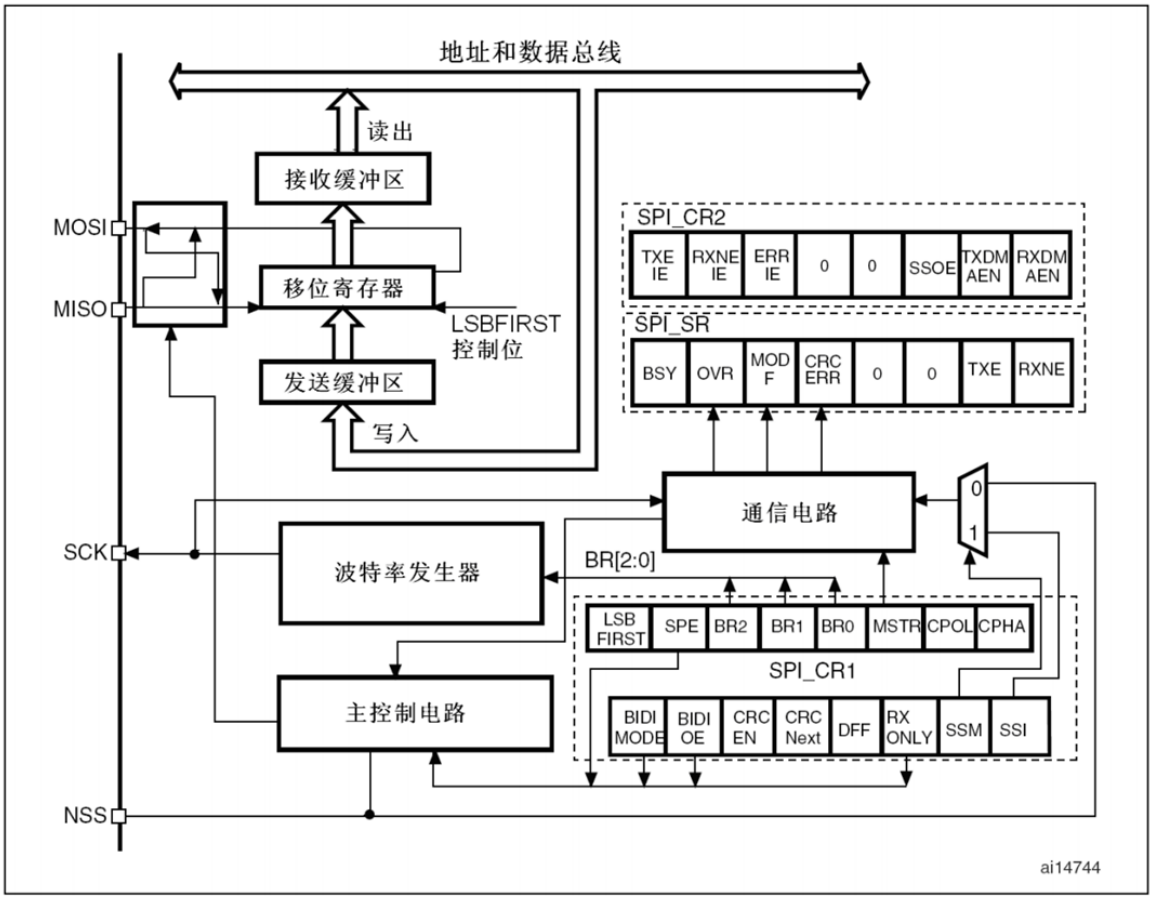
三、SPI外设

1、基本介绍

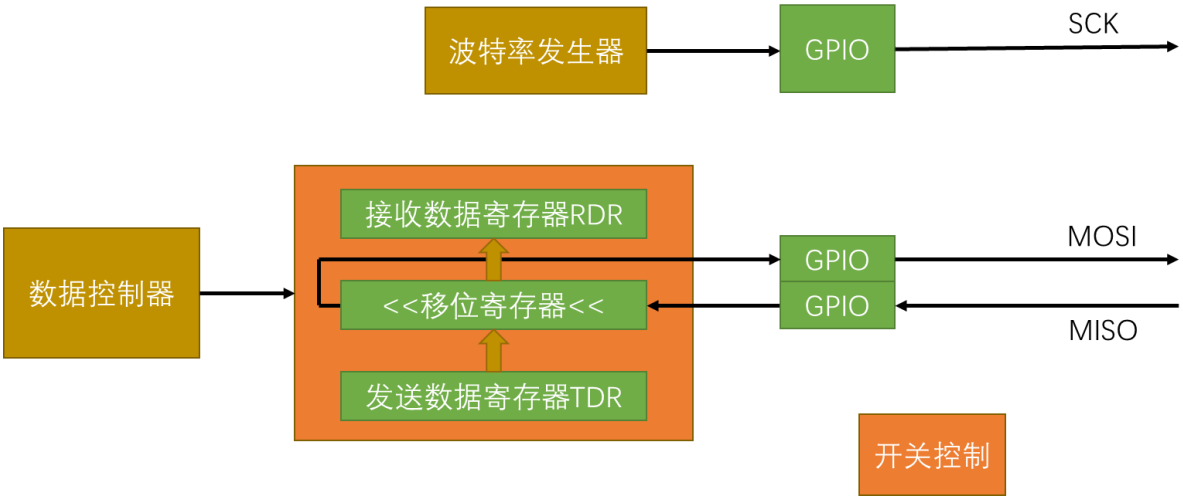
STM32内部集成了硬件SPI收发电路，可以由硬件自动执行时钟生成、数据收发等功能，减轻CPU的负担。可配置8位/16位数据帧、高位先行/低位先行。SPI时钟频率= fPCLK / (2, 4, 8, 16, 32, 64, 128, 256)。SPI的时钟其实就是由PCLK分频得来的，PCLK就是外设时钟，APB2的PCLK就是72MHz，APB1的PCLK是36MHz。比如我们的SPI1是APB2的外设PCLK=72MHz，那它的SPI时钟频率
最大就是只进行二分频，72M / 2=36MHz。像之前I2C的频率最大就只有400K赫兹，所以这里SPI的最大频率比I2C快了90倍。这里频率有些注意事项，一是这个频率数值并不是任意指定的，它只能是PCLK执行这些分频后的数字，就只有上面8个分频选项，最低频率是PCLK的256分频；二是SPI1和SPI2挂载的总线是不一样的，SPI1挂载在APB2上 ，PCLK是72M，SPI2则是挂载在APB1上，PCLK是36M，所以同样的配置，SPI1的使用频率要比SPI2的大一倍。SPI支持多主机模型、主或从操作，可精简为半双工/单工通信，支持DMA，兼容I2S协议（这是一种数字音频信号传输的专用协议，主要应用于音乐数据解码播放）。在STM32F103C8T6芯片中包含的硬件SPI资源有SPI1和SPI2。

2、SPI框图

图209　SPI框图



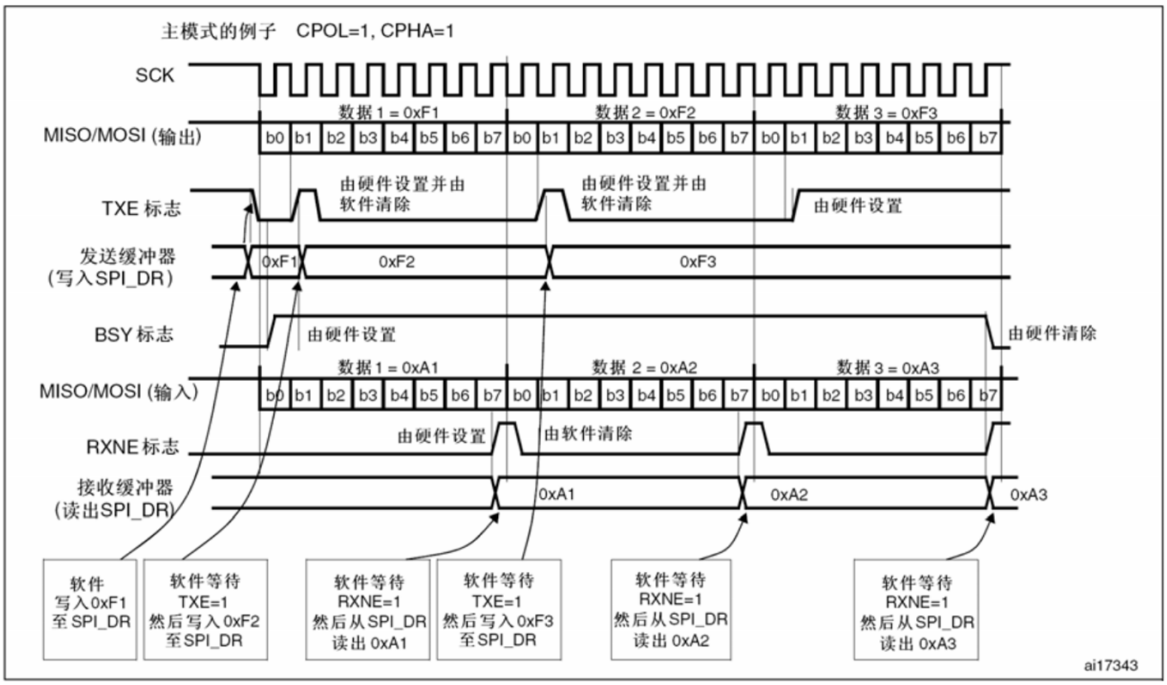
3、SPI基本结构



这是SPI基本结构图，其核心部分就是数据寄存器和移位寄存器。这里发送和接收，分别叫做发送数据寄存器TDR和接收数据寄存器RDR。在上面SPI框图中它又叫发送缓冲区和接收缓冲区。移位寄存器是左移，高位移出去，通过GPIO到MOSI，从MOSI输出，显然这是SPI的主机。之后移入的数据从MISO进来，通过GPIO到移位寄存器的低位，这样循环8次，就能实现主机和从机交换一个字节。然后TDR和RDR的配合，可以实现连续的数据流。此外，TDR数据整体转入移位寄存器的时刻置TxE标志位，移位寄存器数据整体转入RDR的时刻，置RxNE标志位。波特率发生器负责产生时钟，输出到SCK引脚。数据控制器可以就看成是一个管理员，它控制着所有电路的运行。最后开关控制就是SPI_CMD初始化之后给个ENABLE使能整个外设。这个简化结构图中没有画SS从机选择引脚，这个引脚使用普通的GPIO来模拟即可，在一主多从的模型下，GPIO模拟的SS是最佳选择。

4、主模式全双工连续传输

图213　主模式、全双工模式下(BIDIMODE=0并且RXONLY=0)连续传输时，TXE/RXNE/BSY的变化示意图



这是主模式全双工连续传输的时序图。演示的是借助缓冲区数据，前仆后继实现连续数据流的过程，但是这个流程稍微比较复杂，也不太方便封装，所以在实际过程中，如果对性能没有极致的追求，通常更倾向使用下面稍后要讲的非连续传输的示意图。这个非连续传输使用起来更加简单，实际用的话，只需要4行代码就能完成任务了。非连续传输的好处就是容易封装，好理解好用，但是会损失一丢丢性能。连续传输传输更快，但是操作起来相对复杂。

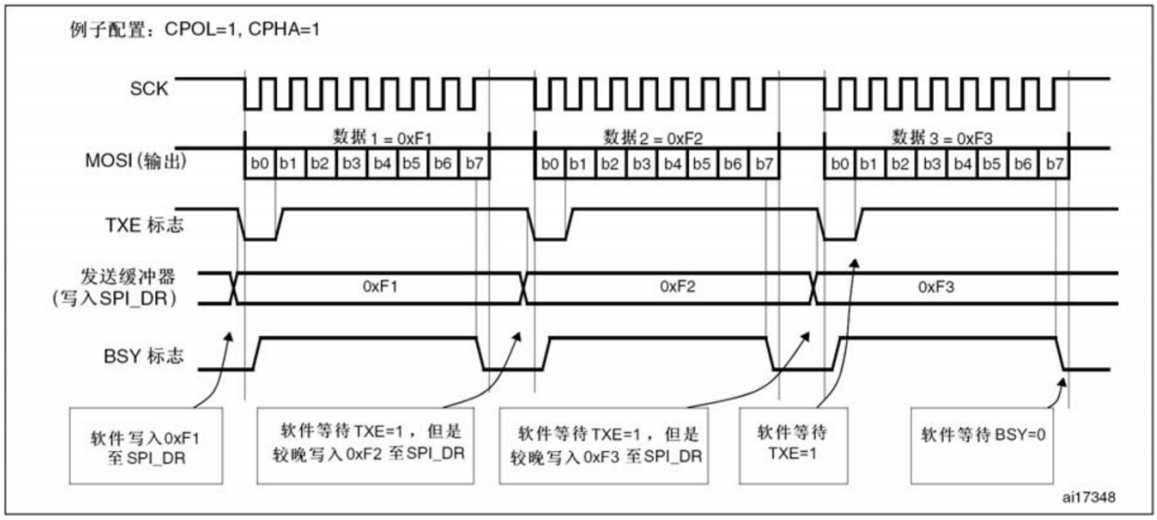
首先第一行是SCK时钟线，这里CPOL=1，CPHA=1，示例使用的是SPI模式3，
所以SCK默认是高电平，然后在第一个下降沿，MOSI和MISO移出数据，之后上升沿移入数据，依次这样来进行。下面第二行是MOSI和MISO输出的波形，跟随SCK时钟变换，数据位依次出现。从前到后依次出现的是b0、b1一直到b7，所以这里示例演示的是低位先行的模式，实际SPI高位先行用的更多一些。之后第三行是TxE发送寄存器，空标志位，波形是这样的，等会再分析。下面就是发送缓冲器，写入SPI_DR，实际上就是TDR。然后BSY，BUSY，是由硬件自动设置和清除的。当有数据传输时，BUSY置1。上面部分就是输出的流程和现象。然后下面是输入的流程和现象。第一个是MISO/MOSI的输入数据，之后是RxNE接收数据寄存器非空标志位，最后是接收缓冲器读出SPI_DR，就是结构框图中的RDR。了解了各个信号的定义，接下来从左到右依次分析。首先SS置低电平，开始时序，这个图中没画，但是是必须得有的。在刚开始时TxE为1，表示TDR空，可以写入数据开始传输，然后下面指示的第一步就是软件写入0xF1至SPI_DR，0xF1就是要发送的第一个数据，最后可以看到写入之后，TDR变为0xF1，同时TxE变为0，表示TDR已经有数据了，此时TDR是等候区，移位寄存器才是真正的发送区，移位寄存器刚开始肯定没有数据2。所以在等候区，TDR里的F1就会立刻转入移位寄存器，开始发送，转入瞬间置TxE标志为1，表示发送寄存器空，然后移位寄存器有数据了，波形就自动开始生成。这样数据转入移位寄存器之后，数据F1的波形就开始产生了。在移位产生F1波形的同时，等候区TDR是空的，未来移位完成时，下一个数据能不间断的跟随。这就要提早把下一个数据写入到TDR等着，所以接下来第二步的操作是写入F1之后，软件等待TxE=1，在这个位置，一旦TDR空了，就写入F2至SPI_DR，写入之后，可以看到TDR的内容就变成F2了。要是把下个数据放到TDR里候着，之后的发送流程也是同理。F1数据波形产生完毕后，F2转入移位寄存器开始发送，这时TxE=1，就需要尽快把下一个数据F3，放到TDR里等着，然后等待TxE=1，
然后写入F3至TDR，写入之后TDR变为F3，最后，在这里如果只向发送3个数据，F3转入移位寄存器之后，TxE=1，就不需要继续写入了，TxE之后一直是1。注意在最后一个TxE=1之后，还需要继续等待一段时间，F3的波形才能完整发送完，等波形全部完整发送之后，BUSY标志由硬件清除，这才表示波形发送完成了。这些就是发送的流程。
继续看一下接收的流程，SPI是全双工，发送的同时还有接收。在第一个字节发送完成后，第一个字节的

接收也完成了。接收到的数据1是A1，这时移位寄存器的数据整体转入RDR，RDR随后存储的就是A1，转入的同时RxNE标志位也置1，表示收到数据了。软件等待RxNE=1，表示收到数据了。然后从SPI_DR也就是RDR读出数据A1，这是第一个接收到的数据。接收之后软件清除RxNE标志位，然后当下一个数据2收到之后，RxNE重新置1，当监测到RxNE等于1时，就继续读出RDR，这是第二个数据A2。最后，在最后一个字节时序完全产生之后
数据3才能收到，所以数据3直到此时才能读出来。注意一个字节波形收到后，移位寄存器的数据自动转入RDR会覆盖原有的数据，所以读出RDR要及时，否则下一个数据A2就会覆盖A1，就不能实现连续数据流的接收了。这就是整个发送和接收的流程。

连续数据流对软件的配合要求较高，在每个标志位产生后，数据都要及时处理。配合的好，时钟可以连续不间断的产生，每个字节之间没有任何空隙，传输效率是最高的。如果你对传输效率有非常高的要求，那就要好好研究这个
连续数据流传输。但是入门的话可以先不用这个，因为这个操作比较复杂，而且数据的位置交叉比较多，所以更推荐下面这个非连续传输。

5、非连续传输

图218　非连续传输发送(BIDIMODE=0并且RXONLY=0)时，TXE/BSY变化示意图



这是非连续传输发送的示意图。这个非连续传输和连续传输有什么区别呢？首先这个配置还是SPI模式3，SCK默认高电平。当想发送数据时，如果检测到TxE=1，TDR为空，就软件写入0xF1至SPI_DR，这时TDR的值变为F1，TxE变为0。目前移位寄存器也是空，所以这个F1会立刻转入移位寄存器，开始发送，波形产生并且TxE置回1，表示可以把下一个数据放在TDR里候着了。但是现在区别就来了，在连续传输情形下，一旦TxE=1，就会把下一个数据写到TDR里候着，这样是为了连续传输数据衔接更紧密。但是这样的话流程就比较混乱，程序写起来比较复杂，所以在非连续传输这里TxE=1了，不着急把下一个数据填进去，而是一直等待，等第一个字节时序结束。此时意味着接收第一个字节也完成了，这时接收的RxNE会置1，等待RxNE置1后，先把第一个接收到的数据读出来，之后再写入下一个字节数据这里的软件等待TxE=1，但是较晚写入0XF2至SPI_DR，较晚写入TDR后，数据2开始发送，还是不着急写数据3，等到了RxNE置1后，先把接收的数据2收着，再继续写入数据3，数据3持续结束后，最后再接收数据3置换回来的数据。这个流程的整个步骤就是第一步等待TxE为1，第二步写入发送的数据至TDR，第三步等待RxNE为1，第四步读取RDR接收的数据，之后交换第二个字节，重复这四步，第一步等待TxE为1，第二步写入发送的数据至TDR，第三步等待RxNE为1，第四步读取RDR接收的数据，之后交换第三个字节，也是同理。这就是非连续传输的流程。

那非连续传输缺点就是没有及时把下一个数据写入TDR候着，所以等到第一个字节时序完成后，第二个字节还没有送过来，这个数据传输就会等着，所以这里时钟和数据的时序在字节与字节之间会产生间隙，拖慢了整体数据传输的速度，这个间隙在SCK频率低的时候，影响不大，但是在SCK频率非常高时，间隙拖后腿的现象就比较严重了。

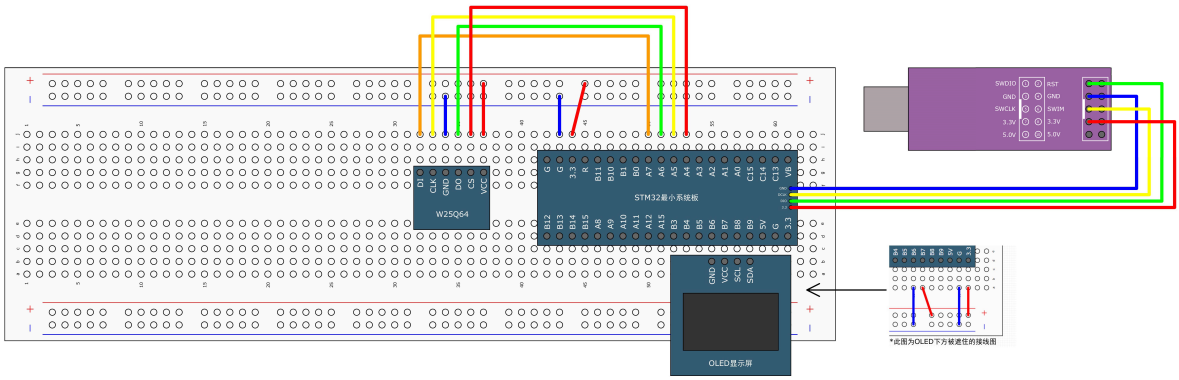
四、SPI试验

1、软件SPI读写W25Q64

（1）实验目的

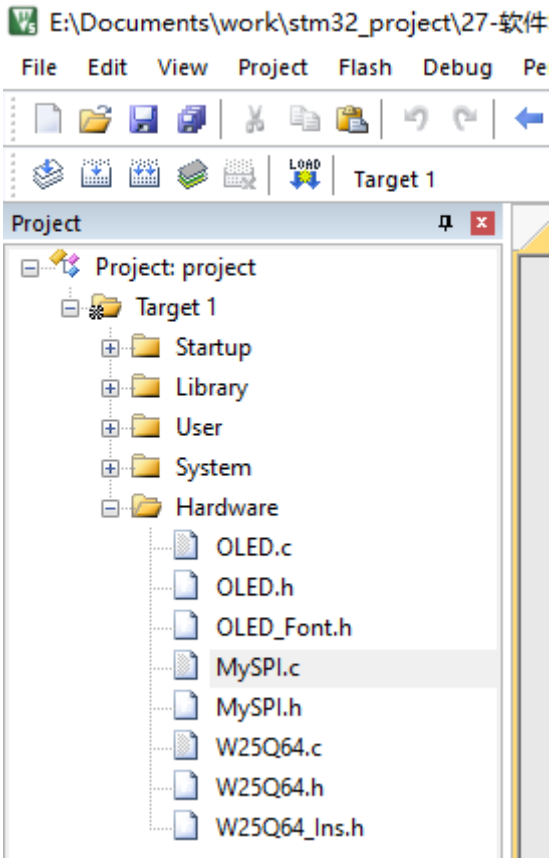
通过控制STM32利用软件实现SPI协议，对W25Q64这一Flash芯片进行读写操作，并在OLED上显示获得的数据值。

（2）硬件电路接线图



（3）程序设计

工程结构如下图：



其中Hardware组与之前工程不一样，这个组主要增加了MySPI、W25Q64的相关驱动代码。具体代码展示如下：

MySPI.c

```
#include "stm32f10x.h"                  // Device header

void MySPI_W_SS(uint8_t BitValue) {
```

```
    // SPI写SS引脚电平
    GPIO_WriteBit(GPIOA, GPIO_Pin_4, (BitAction)BitValue);  // 根据BitValue，设置SS引脚的电平
}

void MySPI_W_SCK(uint8_t BitValue) {
    // SPI写SCK引脚电平
    GPIO_WriteBit(GPIOA, GPIO_Pin_5, (BitAction)BitValue);  // 根据BitValue，设置SCK引脚的电平
}

void MySPI_W_MOSI(uint8_t BitValue) {
    // SPI写MOSI引脚电平
    GPIO_WriteBit(GPIOA, GPIO_Pin_7, (BitAction)BitValue);  // 根据BitValue，设置MOSI引脚的电平
                                                            // BitValue要实现非0即1的特性
}

uint8_t MySPI_R_MISO(void) {
    // SPI读MISO引脚电平
    return GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_6);    // 读取MISO电平并返回
}

void MySPI_Init(void) {
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);   // 打开GPIOA的时钟

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_4 | GPIO_Pin_5 | GPIO_Pin_7;
    GPIO_Init(GPIOA, &GPIO_InitStructure);  // 将PA4、PA5和PA7引脚初始化为推挽输出

    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6;
    GPIO_Init(GPIOA, &GPIO_InitStructure);  // 将PA6引脚初始化为上拉输入

    // 设置默认电平
    MySPI_W_SS(1);      // 配置SS默认高电平
    MySPI_W_SCK(0);     // 配置SCK默认低电平
}

void MySPI_Start(void) {
    // SPI起始
    MySPI_W_SS(0);
}

void MySPI_Stop(void) {
    // SPI终止
    MySPI_W_SS(1);
}
```

```
uint8_t MySPI_SwapByte(uint8_t ByteSend) {
    uint8_t i, ByteReceive;     // 定义接收的数据，并赋初值0x00，此处必须赋初值0x00，后面会用到

    for (i = 0; i < 8; i++) {
        MySPI_W_MOSI(ByteSend & (0x80 >> i));   // 使用掩码的方式取出ByteSend的指定一位数据并写入到MOSI线
        MySPI_W_SCK(1);     // 拉高SCK，上升沿移出数据
        if (MySPI_R_MISO() == 1) {ByteReceive |= (0x80 >> i);}  // 使用掩码的方式取出ByteSend的指定一位数据
                                                                // 并写入到MOSI线
        MySPI_W_SCK(0);     // 拉低SCK，下降沿移入数据
    }

    return ByteReceive;
}

// 与上个方法功能相同的不同实现
//uint8_t MySPI_SwapByte(uint8_t ByteSend) {
//  uint8_t i;
//
//  for (i = 0; i < 8; i++) {
//      MySPI_W_MOSI(ByteSend & 0x80);
//      ByteSend <<= 1;
//      MySPI_W_SCK(1);
//      if (MySPI_R_MISO() == 1) {ByteSend |= 0x01;}
//      MySPI_W_SCK(0);
//  }
//
//  return ByteSend;
//}
```

MySPI.h

```
#ifndef __MYSPI_H
#define __MYSPI_H

void MySPI_Init(void);
void MySPI_Start(void);
void MySPI_Stop(void);
uint8_t MySPI_SwapByte(uint8_t ByteSend);

#endif
```

W25Q64.c

```
#include "stm32f10x.h"                  // Device header
#include "MySPI.h"
```

```c
#include "W25Q64_Ins.h"

void W25Q64_Init(void) {
    MySPI_Init();
}

void W25Q64_ReadID(uint8_t *MID, uint16_t *DID) {
    MySPI_Start();      // SPI起始
    MySPI_SwapByte(W25Q64_JEDEC_ID);        // 交换发送读取ID的指令
    *MID = MySPI_SwapByte(W25Q64_DUMMY_BYTE);       // 交换接收MID，通过输出参数返回
    *DID = MySPI_SwapByte(W25Q64_DUMMY_BYTE);       // 交换接收DID高8位
    *DID <<= 8;     // 高8位移到高位
    *DID |= MySPI_SwapByte(W25Q64_DUMMY_BYTE);      // 或上交换接收DID的低8位，通过输出参数返回
    MySPI_Stop();       // SPI终止
}

void W25Q64_WriteEnable(void) {
    MySPI_Start();      // SPI起始
    MySPI_SwapByte(W25Q64_WRITE_ENABLE);    // 交换发送写使能的指令
    MySPI_Stop();       // SPI终止
}

void W25Q64_WaitBusy(void) {
    uint32_t timeout;
    MySPI_Start();      // SPI起始
    MySPI_SwapByte(W25Q64_READ_STATUS_REGISTER_1);      // 交换发送读状态寄存器1的指令
    timeout = 100000;   // 给定超时计数时间
    // 循环等待忙标志位
    while ((MySPI_SwapByte(W25Q64_DUMMY_BYTE) & 0x01) == 0x01) {
        timeout --;     // 等待时，计数值自减
        // 自减到0后，等待超时
        if (timeout == 0) {
            break;      // 跳出等待，不等了
        }
    }
    MySPI_Stop();       // SPI终止
}

void W25Q64_PageProgram(uint32_t Address, uint8_t *DataArray, uint16_t Count) {
    uint16_t i;
    W25Q64_WriteEnable();       // 写使能

    MySPI_Start();              // SPI起始
    MySPI_SwapByte(W25Q64_PAGE_PROGRAM);    // 交换发送页编程的指令
    MySPI_SwapByte(Address >> 16);          // 交换发送地址23~16位
    MySPI_SwapByte(Address >> 8);           // 交换发送地址15~8位
    MySPI_SwapByte(Address);
    for (i = 0; i < Count; i++) {
        MySPI_SwapByte(DataArray[i]);       // 依次在起始地址后写入数据
    }
```

```
    MySPI_Stop();       // SPI终止

    W25Q64_WaitBusy();      // 等待忙
}

void W25Q64_SectorErase(uint32_t Address) {
    W25Q64_WriteEnable();       // 写使能

    MySPI_Start();          // SPI起始
    MySPI_SwapByte(W25Q64_SECTOR_ERASE_4KB);        // 交换发送扇区擦除的指令
    MySPI_SwapByte(Address >> 16);                  // 交换发送地址23~16位
    MySPI_SwapByte(Address >> 8);                   // 交换发送地址15~8位
    MySPI_SwapByte(Address);                        // 交换发送地址7~0位
    MySPI_Stop();                                   // SPI终止

    W25Q64_WaitBusy();                              // 等待忙
}

void W25Q64_ReadData(uint32_t Address, uint8_t *DataArray, uint32_t Count) {
    uint32_t i;
    MySPI_Start();          // SPI起始
    MySPI_SwapByte(W25Q64_READ_DATA);               // 交换发送读取数据的指令
    MySPI_SwapByte(Address >> 16);                  // 交换发送地址23~16位
    MySPI_SwapByte(Address >> 8);                   // 交换发送地址15~8位
    MySPI_SwapByte(Address);                        // 交换发送地址7~0位
    for (i = 0; i < Count; i++) {
        DataArray[i] = MySPI_SwapByte(W25Q64_DUMMY_BYTE);   // 依次在起始地址后读取数据
    }
    MySPI_Stop();                                   // SPI终止
}
```

W25Q64.h

```
#ifndef __W25Q64_H
#define __W25Q64_H

void W25Q64_Init(void);
void W25Q64_ReadID(uint8_t *MID, uint16_t *DID);
void W25Q64_PageProgram(uint32_t Address, uint8_t *DataArray, uint16_t Count);
void W25Q64_SectorErase(uint32_t Address);
void W25Q64_ReadData(uint32_t Address, uint8_t *DataArray, uint32_t Count);

#endif
```

W25Q64_Ins.h

```
#ifndef __W25Q64_INS_H
#define __W25Q64_INS_H
```

```
// 自定义W25Q64寄存器地址宏定义
#define W25Q64_WRITE_ENABLE                     0x06
#define W25Q64_WRITE_DISABLE                    0x04
#define W25Q64_READ_STATUS_REGISTER_1           0x05
#define W25Q64_READ_STATUS_REGISTER_2           0x35
#define W25Q64_WRITE_STATUS_REGISTER            0x01
#define W25Q64_PAGE_PROGRAM                     0x02
#define W25Q64_QUAD_PAGE_PROGRAM                0x32
#define W25Q64_BLOCK_ERASE_64KB                 0xD8
#define W25Q64_BLOCK_ERASE_32KB                 0x52
#define W25Q64_SECTOR_ERASE_4KB                 0x20
#define W25Q64_CHIP_ERASE                       0xC7
#define W25Q64_ERASE_SUSPEND                    0x75
#define W25Q64_ERASE_RESUME                     0x7A
#define W25Q64_POWER_DOWN                       0xB9
#define W25Q64_HIGH_PERFORMANCE_MODE            0xA3
#define W25Q64_CONTINUOUS_READ_MODE_RESET       0xFF
#define W25Q64_RELEASE_POWER_DOWN_HPM_DEVICE_ID 0xAB
#define W25Q64_MANUFACTURER_DEVICE_ID           0x90
#define W25Q64_READ_UNIQUE_ID                   0x4B
#define W25Q64_JEDEC_ID                         0x9F
#define W25Q64_READ_DATA                        0x03
#define W25Q64_FAST_READ                        0x0B
#define W25Q64_FAST_READ_DUAL_OUTPUT            0x3B
#define W25Q64_FAST_READ_DUAL_IO                0xBB
#define W25Q64_FAST_READ_QUAD_OUTPUT            0x6B
#define W25Q64_FAST_READ_QUAD_IO                0xEB
#define W25Q64_OCTAL_WORD_READ_QUAD_IO          0xE3

#define W25Q64_DUMMY_BYTE                       0xFF

#endif
```

main.c

```
#include "stm32f10x.h"                  // Device header
#include "OLED.h"
#include "W25Q64.h"

uint8_t MID;        // 定义用于存放MID号的变量
uint16_t DID;       // 定义用于存放DID号的变量

uint8_t ArrayWrite[] = {0x01, 0x02, 0x03, 0x04};    // 定义要写入数据的测试数组
uint8_t ArrayRead[4];       // 定义要读取数据的测试数组

int main(void){

    OLED_Init();
    W25Q64_Init();
```

```
    OLED_ShowString(1, 1, "MID:  DID:");
    OLED_ShowString(2, 1, "W:");
    OLED_ShowString(3, 1, "R:");

    W25Q64_ReadID(&MID, &DID);       // 获取W25Q64的ID号
    OLED_ShowHexNum(1, 5, MID, 2);
    OLED_ShowHexNum(1, 12, DID, 4);

    W25Q64_SectorErase(0x000000);   // 扇区擦除
    W25Q64_PageProgram(0x000000, ArrayWrite, 4);     // 将写入数据的测试数组写入到W25Q64中

    W25Q64_ReadData(0x000000, ArrayRead, 4);     // 读取刚写入的测试数据到读取数据的测试数组中

    // 显示写入的数据
    OLED_ShowHexNum(2, 3, ArrayWrite[0], 2);
    OLED_ShowHexNum(2, 6, ArrayWrite[1], 2);
    OLED_ShowHexNum(2, 9, ArrayWrite[2], 2);
    OLED_ShowHexNum(2, 12, ArrayWrite[3], 2);

    // 显示读取的数据
    OLED_ShowHexNum(3, 3, ArrayRead[0], 2);
    OLED_ShowHexNum(3, 6, ArrayRead[1], 2);
    OLED_ShowHexNum(3, 9, ArrayRead[2], 2);
    OLED_ShowHexNum(3, 12, ArrayRead[3], 2);

    while(1){

    }
}
```

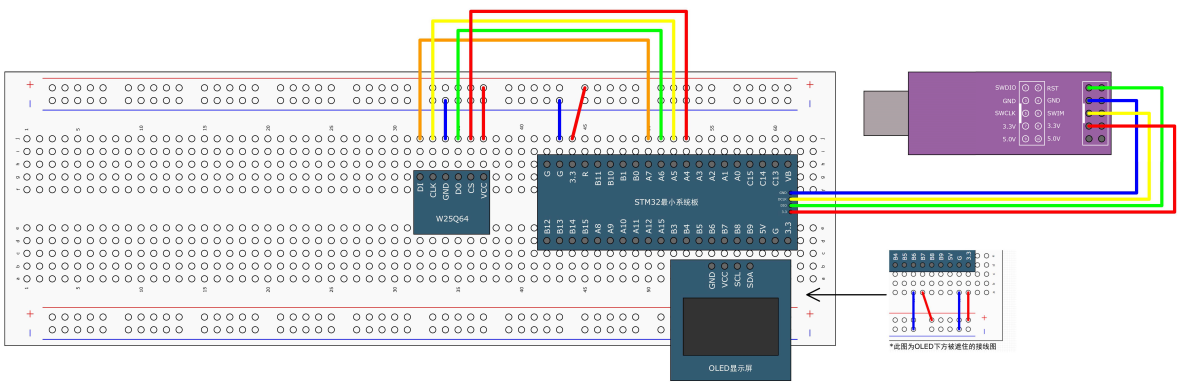
（4）代码完成后，编译并下载到硬件中执行，会发现实验结果是W25Q64的芯片ID以及存储的数据都能够正确读取并显示到OLED上，符合预期设计。

2、硬件SPI读写W25Q64

（1）实验目的

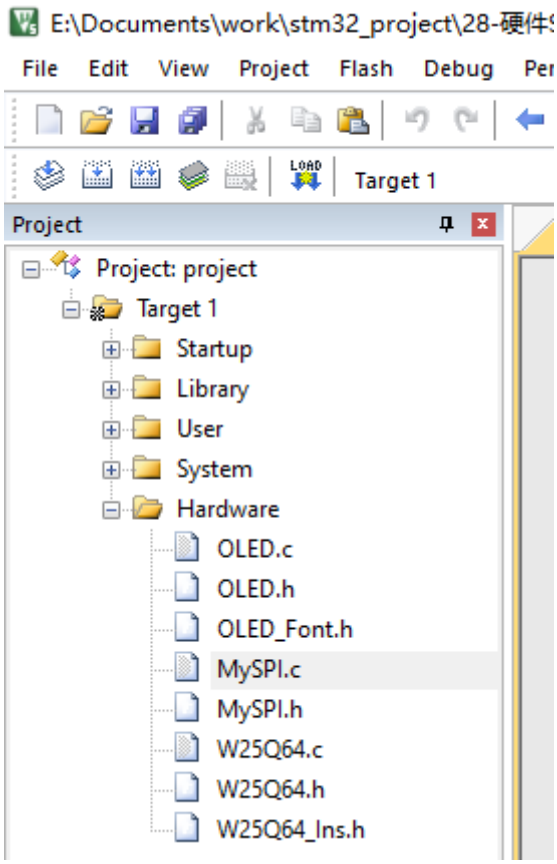
通过控制STM32利用其硬件SPI，对W25Q64这一Flash芯片进行读写操作，并在OLED上显示获得的数据值。

（2）硬件电路接线图



（3）程序设计

工程结构如下图：



其中Hardware组与之前工程相比，这个组主要调整了MySPI、W25Q64的相关驱动代码。具体代码展示如下：

MySPI.c

```
#include "stm32f10x.h"                  // Device header

void MySPI_W_SS(uint8_t BitValue) {
    GPIO_WriteBit(GPIOA, GPIO_Pin_4, (BitAction)BitValue);
}

void MySPI_Init(void) {
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);   // 打开GPIOA的时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_SPI1, ENABLE);    // 打开SPI1的时钟

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;    // 配置为通用推挽输出
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_4;       // SS
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;     // 配置为复用推挽输出
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_5 | GPIO_Pin_7;      // SCK和MOSI
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;       // 配置为上拉输入
```

```
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6;       // MISO
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    SPI_InitTypeDef SPI_InitStructure;
    SPI_InitStructure.SPI_BaudRatePrescaler = SPI_BaudRatePrescaler_128;    // 波特率预分频器，72m/128≈500多KHz
    // 配置SPI模式0
    SPI_InitStructure.SPI_CPHA = SPI_CPHA_1Edge;    // CPHA = 0
    SPI_InitStructure.SPI_CPOL = SPI_CPOL_Low;      // CPOL = 0
    SPI_InitStructure.SPI_CRCPolynomial = 7;        // CRC校验多项式，一般不用，这里选择默认值7
    SPI_InitStructure.SPI_DataSize = SPI_DataSize_8b;   // 配置数据宽度8位
    SPI_InitStructure.SPI_Direction = SPI_Direction_2Lines_FullDuplex;  // 配置使用双线全双工
    SPI_InitStructure.SPI_FirstBit = SPI_FirstBit_MSB;      // 配置高位先行
    SPI_InitStructure.SPI_Mode = SPI_Mode_Master;   // 配置主模式
    SPI_InitStructure.SPI_NSS = SPI_NSS_Soft;       // 计划选择GPIO模拟，NSS暂时不用
    SPI_Init(SPI1, &SPI_InitStructure);

    SPI_Cmd(SPI1, ENABLE);      // 使能SPI

    MySPI_W_SS(1);      // 给SS高电平，默认不选择从机
}

void MySPI_Start(void) {
    MySPI_W_SS(0);
}

void MySPI_Stop(void) {
    MySPI_W_SS(1);
}

uint8_t MySPI_SwapByte(uint8_t ByteSend) {
    while (SPI_I2S_GetFlagStatus(SPI1, SPI_I2S_FLAG_TXE) != SET);   // 等待TXE等于1
    SPI_I2S_SendData(SPI1, ByteSend);       // 写发送的数据至TDR
    while (SPI_I2S_GetFlagStatus(SPI1, SPI_I2S_FLAG_RXNE) != SET);  // 等待RXNE等于1，发送完成即接收完成
    return SPI_I2S_ReceiveData(SPI1);       // 从RDR读取接收的数据
}
```

MySPI.h

```h
#ifndef __MYSPI_H
#define __MYSPI_H

void MySPI_Init(void);
void MySPI_Start(void);
void MySPI_Stop(void);
uint8_t MySPI_SwapByte(uint8_t ByteSend);

#endif
```

W25Q64.c

```c
#include "stm32f10x.h"                  // Device header
#include "MySPI.h"
#include "W25Q64_Ins.h"

void W25Q64_Init(void) {
    MySPI_Init();
}

void W25Q64_ReadID(uint8_t *MID, uint16_t *DID) {
    MySPI_Start();          // SPI起始
    MySPI_SwapByte(W25Q64_JEDEC_ID);        // 交换发送读取ID的指令
    *MID = MySPI_SwapByte(W25Q64_DUMMY_BYTE);   // 交换接收MID，通过输出参数返回
    *DID = MySPI_SwapByte(W25Q64_DUMMY_BYTE);   // 交换接收DID高8位
    *DID <<= 8;                                 // 高8位移到高位
    *DID |= MySPI_SwapByte(W25Q64_DUMMY_BYTE);  // 或上交换接收DID的低8位，通过输出参数返回
    MySPI_Stop();           // SPI终止
}

void W25Q64_WriteEnable(void) {
    MySPI_Start();          // SPI起始
    MySPI_SwapByte(W25Q64_WRITE_ENABLE);    // 交换发送写使能的指令
    MySPI_Stop();           // SPI终止
}

void W25Q64_WaitBusy(void) {
    uint32_t timeout;
    MySPI_Start();          // SPI起始
    MySPI_SwapByte(W25Q64_READ_STATUS_REGISTER_1);      // 交换发送读状态寄存器1的指令
    timeout = 100000;       // 配置超时计数时间
    while ((MySPI_SwapByte(W25Q64_DUMMY_BYTE) & 0x01) == 0x01) {
        timeout --;
        // 自减到0后，等待超时
        if (timeout == 0) {
            break;
        }
    }
    MySPI_Stop();           // SPI终止
}
```

```
void W25Q64_PageProgram(uint32_t Address, uint8_t *DataArray, uint16_t Count) {
    uint16_t i;
    W25Q64_WriteEnable();       // 写使能

    MySPI_Start();          // SPI起始
    MySPI_SwapByte(W25Q64_PAGE_PROGRAM);    // 交换发送页编程的指令
    MySPI_SwapByte(Address >> 16);      // 交换发送地址23~16位
    MySPI_SwapByte(Address >> 8);       // 交换发送地址15~8位
    MySPI_SwapByte(Address);            // 交换发送地址7~0位
    for (i = 0; i < Count; i++) {
        MySPI_SwapByte(DataArray[i]);       // 依次在起始地址后写入数据
    }
    MySPI_Stop();           // SPI终止

    W25Q64_WaitBusy();      // 等待忙
}

void W25Q64_SectorErase(uint32_t Address) {
    W25Q64_WriteEnable();       // 写使能

    MySPI_Start();              // SPI起始
    MySPI_SwapByte(W25Q64_SECTOR_ERASE_4KB);    // 交换发送扇区擦除的指令
    MySPI_SwapByte(Address >> 16);      // 交换发送地址23~16位
    MySPI_SwapByte(Address >> 8);       // 交换发送地址15~8位
    MySPI_SwapByte(Address);            // 交换发送地址7~0位
    MySPI_Stop();               // SPI终止

    W25Q64_WaitBusy();          // 等待忙
}

void W25Q64_ReadData(uint32_t Address, uint8_t *DataArray, uint32_t Count) {
    uint32_t i;
    MySPI_Start();              // SPI起始
    MySPI_SwapByte(W25Q64_READ_DATA);       // 交换发送读取数据的指令
    MySPI_SwapByte(Address >> 16);          // 交换发送地址23~16位
    MySPI_SwapByte(Address >> 8);           // 交换发送地址15~8位
    MySPI_SwapByte(Address);                // 交换发送地址7~0位
    for (i = 0; i < Count; i++) {
        DataArray[i] = MySPI_SwapByte(W25Q64_DUMMY_BYTE);   // 依次在起始地址后读取数据
    }
    MySPI_Stop();               // SPI终止
}
```

W25Q64.h

```
#ifndef __W25Q64_H
#define __W25Q64_H

void W25Q64_Init(void);
void W25Q64_ReadID(uint8_t *MID, uint16_t *DID);
void W25Q64_PageProgram(uint32_t Address, uint8_t *DataArray, uint16_t Count);
void W25Q64_SectorErase(uint32_t Address);
void W25Q64_ReadData(uint32_t Address, uint8_t *DataArray, uint32_t Count);

#endif
```

W25Q64_Ins.h

```
#ifndef __W25Q64_INS_H
#define __W25Q64_INS_H

// 自定义W25Q64寄存器地址宏定义
#define W25Q64_WRITE_ENABLE                     0x06
#define W25Q64_WRITE_DISABLE                    0x04
#define W25Q64_READ_STATUS_REGISTER_1           0x05
#define W25Q64_READ_STATUS_REGISTER_2           0x35
#define W25Q64_WRITE_STATUS_REGISTER            0x01
#define W25Q64_PAGE_PROGRAM                     0x02
#define W25Q64_QUAD_PAGE_PROGRAM                0x32
#define W25Q64_BLOCK_ERASE_64KB                 0xD8
#define W25Q64_BLOCK_ERASE_32KB                 0x52
#define W25Q64_SECTOR_ERASE_4KB                 0x20
#define W25Q64_CHIP_ERASE                       0xC7
#define W25Q64_ERASE_SUSPEND                    0x75
#define W25Q64_ERASE_RESUME                     0x7A
#define W25Q64_POWER_DOWN                       0xB9
#define W25Q64_HIGH_PERFORMANCE_MODE            0xA3
#define W25Q64_CONTINUOUS_READ_MODE_RESET       0xFF
#define W25Q64_RELEASE_POWER_DOWN_HPM_DEVICE_ID 0xAB
#define W25Q64_MANUFACTURER_DEVICE_ID           0x90
#define W25Q64_READ_UNIQUE_ID                   0x4B
#define W25Q64_JEDEC_ID                         0x9F
#define W25Q64_READ_DATA                        0x03
#define W25Q64_FAST_READ                        0x0B
#define W25Q64_FAST_READ_DUAL_OUTPUT            0x3B
#define W25Q64_FAST_READ_DUAL_IO                0xBB
#define W25Q64_FAST_READ_QUAD_OUTPUT            0x6B
#define W25Q64_FAST_READ_QUAD_IO                0xEB
#define W25Q64_OCTAL_WORD_READ_QUAD_IO          0xE3

#define W25Q64_DUMMY_BYTE                       0xFF

#endif
```

main.c

```c
#include "stm32f10x.h"                  // Device header
#include "OLED.h"
#include "W25Q64.h"

uint8_t MID;
uint16_t DID;

uint8_t ArrayWrite[] = {0x01, 0x02, 0x03, 0x04};
uint8_t ArrayRead[4];

int main(void){

    OLED_Init();
    W25Q64_Init();

    OLED_ShowString(1, 1, "MID:  DID:");
    OLED_ShowString(2, 1, "W:");
    OLED_ShowString(3, 1, "R:");

    W25Q64_ReadID(&MID, &DID);      // 获取W25Q64的ID号
    OLED_ShowHexNum(1, 5, MID, 2);
    OLED_ShowHexNum(1, 12, DID, 4);

    W25Q64_SectorErase(0x000000);   // 扇区擦除
    W25Q64_PageProgram(0x000000, ArrayWrite, 4);    // 将写入数据的测试数组写入到W25Q64中

    W25Q64_ReadData(0x000000, ArrayRead, 4);    // 读取刚写入的测试数据到读取数据的测试数组中

    // 显示写入的数据
    OLED_ShowHexNum(2, 3, ArrayWrite[0], 2);
    OLED_ShowHexNum(2, 6, ArrayWrite[1], 2);
    OLED_ShowHexNum(2, 9, ArrayWrite[2], 2);
    OLED_ShowHexNum(2, 12, ArrayWrite[3], 2);

    // 显示读取的数据
    OLED_ShowHexNum(3, 3, ArrayRead[0], 2);
    OLED_ShowHexNum(3, 6, ArrayRead[1], 2);
    OLED_ShowHexNum(3, 9, ArrayRead[2], 2);
    OLED_ShowHexNum(3, 12, ArrayRead[3], 2);

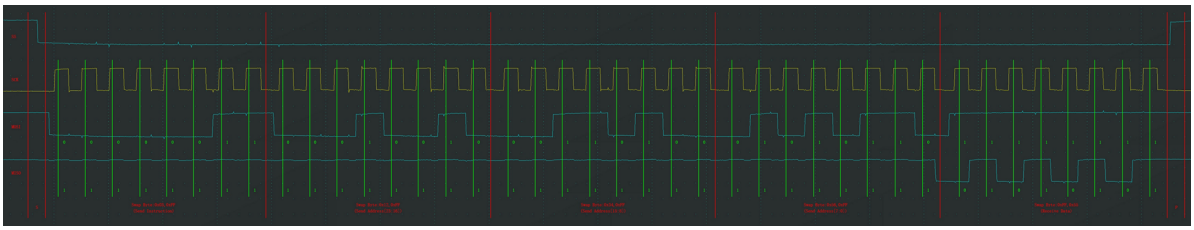
    while(1){

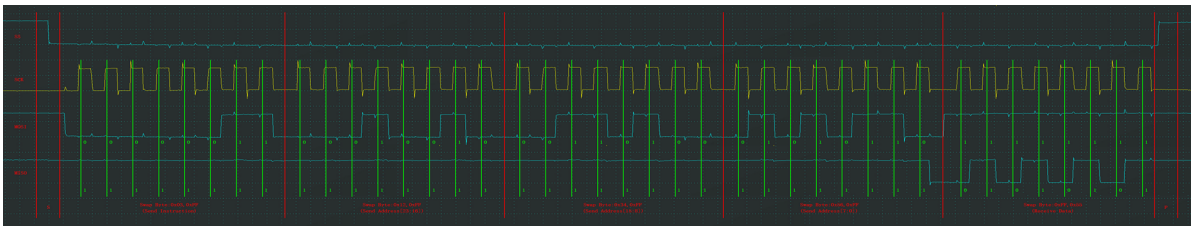
    }
}
```

（4）代码完成后，编译并下载到硬件中执行，会发现实验结果是W25Q64的芯片ID以及存储的数据都能够正确读取并显示到OLED上，符合预期设计。

五、总结

简单看一下软硬件波形对比。





这里上面是软件波形，下面是硬件波形，这些和IC的软硬件波形对比其实都是差不多的。首先，它们的数据变化趋势肯定是一样的采样得到的数据也是一样的。区别就是硬件波形数据线的变化是紧贴SCK边沿的，而软件波形数据线的变化，在边沿后有一些延迟。实际上还可以发现，I2C所描述的SCL低电平期间数据变化，高电平期间数据采样，和SPI所描述的SCK下降沿数据移出，上升沿数据移入，最终波形的表现形式都是一样的。无论是下降沿变化，还是低电平期间变化，都是一个意思。下降沿和低电平期间都可以作为数据变化的时刻，只是硬件波形一般会紧贴边缘，软件波形一般只能在电平期间。当然无论是哪种方式，最终都不会影响数据传输。不过软件波形如果能贴近边缘，还是尽可能贴近边缘，否则如果等太久，比较靠近下一个边缘，数据也容易出错。

课程小结

1. SPI通信协议是一种高速的全双工通信协议，它具有MISO、MOSI、时钟CLK以及片选SS四条通道。可以同时发出和接收数据，在主从模式下无需寻址，而是直接通过片选信号选择从机，提高效率。
2. SPI依据CPOL和CPHA的电平组合配置，可以拥有四种交换字节数据的模式，对于这四种模式是需要重点理解的。
3. 对于W25Qxx系列的Flash存储器的地址分配，写入和读取的特点需要掌握。
4. STM32的SPI基本结构需要重点理解，而对于全双工连续传输模式和非连续传输模式需要理解其差别，优先重点掌握非连续传输模式的使用。

随堂作业

请谈谈STM32的SPI外设基本结构原理。